

Internship Acceptance Task

Asset Management System

A module of the DarkAtlas Attack Surface Monitoring platform

Tracks: Backend Engineering · AI Applications **Duration:** 1 week **Stack:** Python · FastAPI · PostgreSQL

1. Overview

DarkAtlas is Buguard's Attack Surface Monitoring (ASM) platform. It continuously discovers and tracks an organization's internet-facing assets — domains, subdomains, IP addresses, exposed services, TLS certificates, and the technologies running on them — so security teams can see and shrink their external attack surface.

The **Asset Management system** is the module at the heart of that platform: the system of record that ingests discovered assets, removes duplicates, tracks each asset's lifecycle and relationships, and exposes everything for querying, analysis, and reporting.

For this task you will build a self-contained slice of that module. You are **not** expected to build scanners or integrate with the live DarkAtlas platform — focus on modeling, storing, and working with asset data well. Read the shared sections (3, 6, 7), then the section for your track.

2. Who this is for

- **Backend Engineering** — you build the full Asset Management API (Section 4).
- **AI Applications** — you build a minimal API plus a LangChain-powered analysis layer over the asset data (Section 5).

Both tracks share the same domain model and dataset.

3. The core domain: the asset model

Asset types in scope: domain, subdomain, ip_address, service (port + protocol + banner), certificate, technology. Every asset should at minimum capture the following:

Field	Type	Description
id	string / uuid	Unique, stable identifier for the asset.
type	enum	One of: domain, subdomain, ip_address, service, certificate, technology.
value	string	Canonical value, e.g. api.example.com, 203.0.113.10, 443/tcp.
status	enum	active stale archived — drives lifecycle handling.
first_seen	datetime	Set once, when the asset is first recorded.
last_seen	datetime	Updated every time the asset is re-sighted.
source	string	Where it came from: import, scan, manual.
tags	list<string>	Free-form labels for filtering and grouping.

Field	Type	Description
metadata	json	Type-specific fields: cert issuer/expiry, service banner, tech version, etc.

Relationships (the relationships graph): model the links between assets — subdomain → domain, service → ip_address, ip_address ↔ subdomain (resolution), certificate → domain/subdomain, technology → subdomain/service. A single asset can have many relationships.

4. Track A — Backend Engineering (full API)

Build a complete REST API for the Asset Management module.

Mandatory

- **CRUD** for assets — create, read, update, delete.
- **List endpoint** with filtering (by type, status, tag, value contains), sorting, and pagination.
- **Bulk import endpoint** that ingests the provided sample dataset.
- **Deduplication on ingest** — re-importing the same asset must not create a duplicate; it should update last_seen and merge metadata/tags.
- **Lifecycle handling** — set first_seen on creation, update last_seen on every re-sighting, and expose a way to mark assets stale.
- **Relationships** — endpoints to create/read relationships and to fetch an asset together with its related assets (the graph around it).
- **Tagging and search.**
- **Lightweight authentication** (API key or JWT) on write operations.
- **Validation & errors** — input validation and consistent, well-structured error responses.
- **Tests** — automated tests for the core logic (dedup, filtering, relationships at minimum).
- **Docs & infra** — OpenAPI/Swagger (FastAPI gives this for free) and a docker-compose bringing up the API + PostgreSQL.

Optional

- Add one LangChain-powered feature from Track B as a bonus.

5. Track B — AI Applications (minimal API + LangChain)

Build a **small** API — just enough to load the sample dataset and expose your analysis features — and a **mandatory LangChain layer** providing all four capabilities below:

1. **Natural-language asset query** — a user asks in plain English (“show me all expired certificates on production subdomains”); your system translates that into a structured query over the assets and returns the matches.
2. **Risk scoring & summarization** — given an asset or group, produce a risk assessment and a concise summary (expired/expiring certs, sensitive exposed services, end-of-life technologies).
3. **Automated enrichment & categorization** — given a raw or newly imported asset, classify it (environment: prod/staging/dev, category, criticality) and enrich its metadata.

4. **Natural-language report generation** — generate a readable inventory/risk report over the dataset or a filtered subset.

Rules: you must use LangChain (prompt templates, chains, and/or agents) for the analysis layer. The LLM provider is your choice (Anthropic, OpenAI, or a local model); read keys from environment variables and never commit them. Handle invalid input and model errors gracefully, and **guard against the model inventing assets that aren't in the data**.

Minimal API expectation: at least an import endpoint and one endpoint per analysis capability (or a single /analyze endpoint with a mode). Persistence in PostgreSQL is expected, but the API surface stays small.

6. Provided materials

You will receive a **sample asset dataset** as JSON — a representative export from a DarkAtlas scan. Seed your system through the bulk import endpoint. No live scanning or discovery is required. A short excerpt is shown in Appendix A.

7. Edge cases to handle

Part of what we evaluate is how you handle the messy reality of asset data. Please consider:

- **Idempotent imports** — importing the same dataset twice must not create duplicates.
- **Conflicting data** — the same asset from two sources with different metadata; choose a sensible merge strategy.
- **Re-appearing assets** — a stale asset seen again should return to active.
- **Malformed/partial records** in import — fail gracefully; don't crash the whole batch.
- **Large lists** — pagination and sane defaults so a big inventory doesn't return everything at once.
- **Certificate/lifecycle dates** — expired vs. expiring-soon handling.
- **(AI track)** ambiguous or out-of-scope natural-language queries, and hallucination — answers must be grounded in the actual data.
- **(Bonus)** multi-tenant isolation — one organization's assets must never leak into another's view.

State any assumptions you make in your README.

8. Deliverables & submission

Submit a **GitHub repository** containing:

- Source code for your track.
- **docker-compose.yml** that starts everything (app + PostgreSQL) with a single command.
- **A README** covering: setup and run instructions, environment variables, your design decisions and assumptions, API documentation (or a link to the auto-generated docs), and how to run the tests. AI track: include example prompts and their outputs.

No demo video is required. Deadline: **one week** from the date you receive this task. If you run short on time, prioritize the mandatory items and document what you would do next.

9. Evaluation rubric

Each track is scored out of 100. Bonus items can add up to 10 points above the mandatory score (capped at 100).

Track A — Backend Engineering

Criterion	Points	Weight
API correctness & completeness — CRUD, filtering, sorting, pagination	25	Core
ASM features — deduplication, lifecycle, relationships graph	25	Core
Data modeling & database design — schema, migrations, indexing	15	Core
Code quality & structure — readability, separation, error handling	15	Core
Testing — coverage of core logic and edge cases	10	Core
API design & docs — REST conventions, validation, OpenAPI, auth	10	Core
Bonus — multi-tenancy, CI, a LangChain feature, thoughtful extras	+10	Bonus

Track B — AI Applications

Criterion	Points	Weight
LangChain analysis features — all four working (10 each)	40	Core
LLM integration quality — prompting, structured output, grounding, guardrails	20	Core
Minimal API & data integration — clean ingest and persistence	10	Core
Data modeling & handling	10	Core
Code quality & structure	10	Core
README & reproducibility — example prompts/outputs, clear setup	10	Core
Bonus — agentic tool-use, output evaluation, caching, multi-tenancy	+10	Bonus

Across both tracks we also note: clear git history, sensible documented assumptions, and a security-aware mindset — this is a security product, so validate inputs, don't log secrets, and think about abuse.

10. Bonus / stretch goals

- Multi-tenant scoping (organization isolation) across the data model and API.
- Authentication & role-based access control.
- A simple visualization of the asset relationship graph.
- CI pipeline (e.g. GitHub Actions) running your tests.
- Caching and/or rate limiting.
- **Backend:** implement one of the Track B LangChain features.
- **AI:** turn the analysis layer into an agent that calls your own API as tools; add an evaluation harness for output quality.

11. Ground rules

- **AI coding assistants are allowed and expected** — but you must understand and be able to explain every line you submit. We may ask in the interview.
- Use the suggested stack (Python · FastAPI · PostgreSQL) unless you have a good reason to deviate — if so, justify it briefly in your README.
- Keep secrets out of the repo. Use a `.env.example` for configuration.
- Ask questions if anything is unclear; reasonable, clearly documented assumptions are fine.

Appendix A — sample dataset (excerpt)

Illustrative shape only; the dataset you receive will be larger and may contain imperfect records (see Section 7).

```
[
  {
    "id": "a1", "type": "domain", "value": "example.com",
    "status": "active", "source": "scan",
    "tags": ["root"], "metadata": {}
  },
  {
    "id": "a2", "type": "subdomain", "value": "api.example.com",
    "status": "active", "source": "scan", "tags": ["prod"],
    "metadata": {}, "parent": "a1"
  },
  {
    "id": "a3", "type": "certificate", "value": "CN=api.example.com",
    "status": "active", "source": "scan", "tags": [],
    "metadata": { "issuer": "Let's Encrypt", "expires": "2025-01-02" },
    "covers": "a2"
  }
]
```